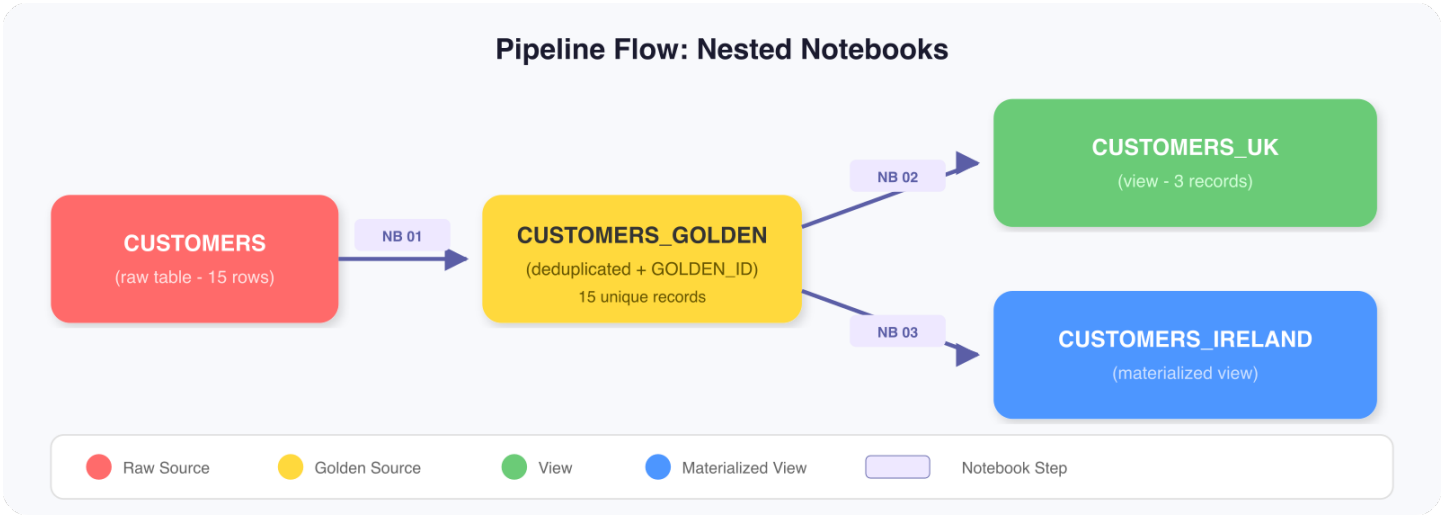


# Nested Notebooks Demo

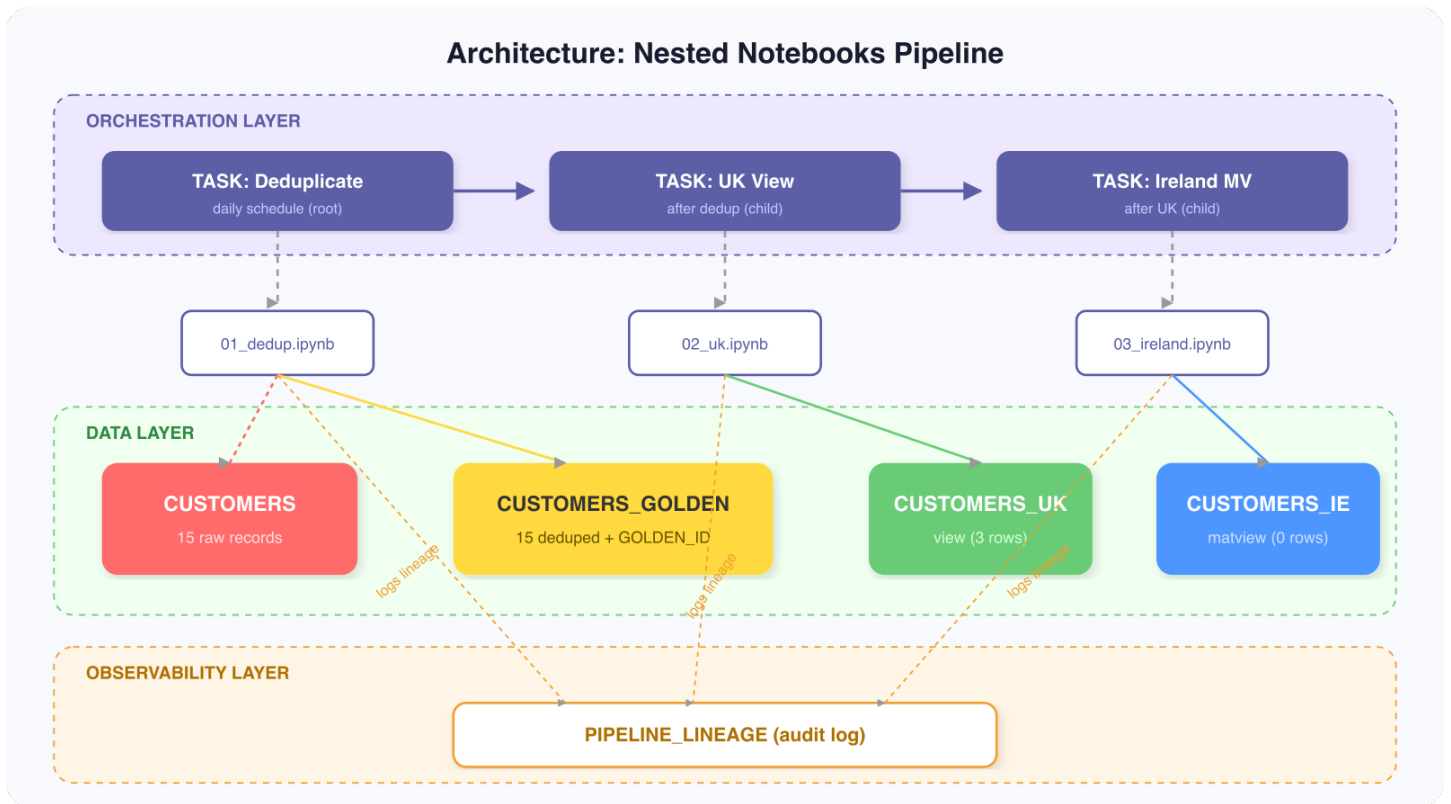
Customer Deduplication Pipeline with Lineage Tracking

Snowflake Tasks | Workspace Notebooks | Pipeline Lineage



# Architecture

This pipeline uses three layers: an Orchestration Layer (Snowflake Tasks), a Data Layer (tables and views), and an Observability Layer (lineage tracking).



# 1. The Problem

*"Customer data often has duplicates — same person, multiple records, slightly different emails or entry dates. Before we can build reliable regional views, we need a single golden source of truth."*

Show the CUSTOMERS table and identify duplicates by email:

```
SELECT EMAIL, COUNT(*) AS cnt
FROM LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS
WHERE EMAIL IS NOT NULL
GROUP BY EMAIL
ORDER BY cnt DESC;
```

## 2. The Solution: Notebook-Driven Pipeline

*"We've built three notebooks that form a pipeline. Each one does one job, logs its lineage, and the whole thing is orchestrated by Snowflake Tasks."*

### Notebooks

- 01\_deduplicate\_customers — Groups by email, assigns GOLDEN\_ID, picks most recent as master
- 02\_customers\_uk\_view — Creates a view of UK master records
- 03\_customers\_ireland\_matview — Creates a materialized view of Ireland master records

### Key Design Decisions

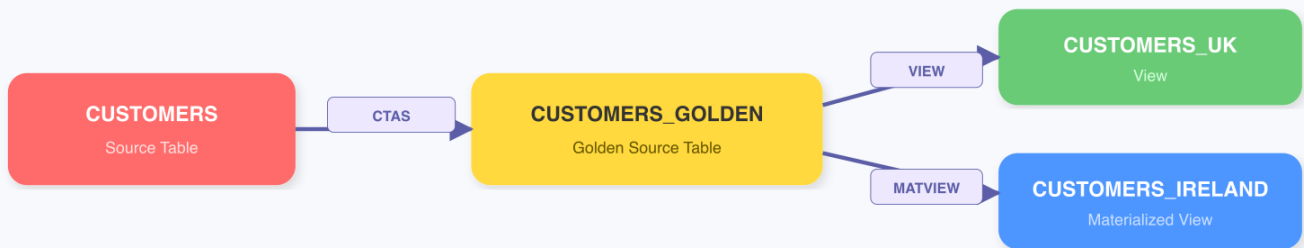
- DENSE\_RANK over email gives a stable golden ID
- IS\_MASTER\_RECORD flag keeps all records but marks the canonical one
- Views sit on top of the golden table — no data duplication

### 3. Lineage Tracking

*"Every notebook logs its source, target, operation, and row count to a lineage table. This gives us an auditable record of what happened, when, and where the data flowed."*

```
SELECT SOURCE_OBJECT AS "FROM", TARGET_OBJECT AS "TO",
       OPERATION, NOTEBOOK_NAME, ROW_COUNT, EXECUTION_TIMESTAMP
FROM LANSDOWNEPARTNERS_DB.CORE.PIPELINE_LINEAGE
ORDER BY EXECUTION_TIMESTAMP DESC;
```

#### Lineage Tracking

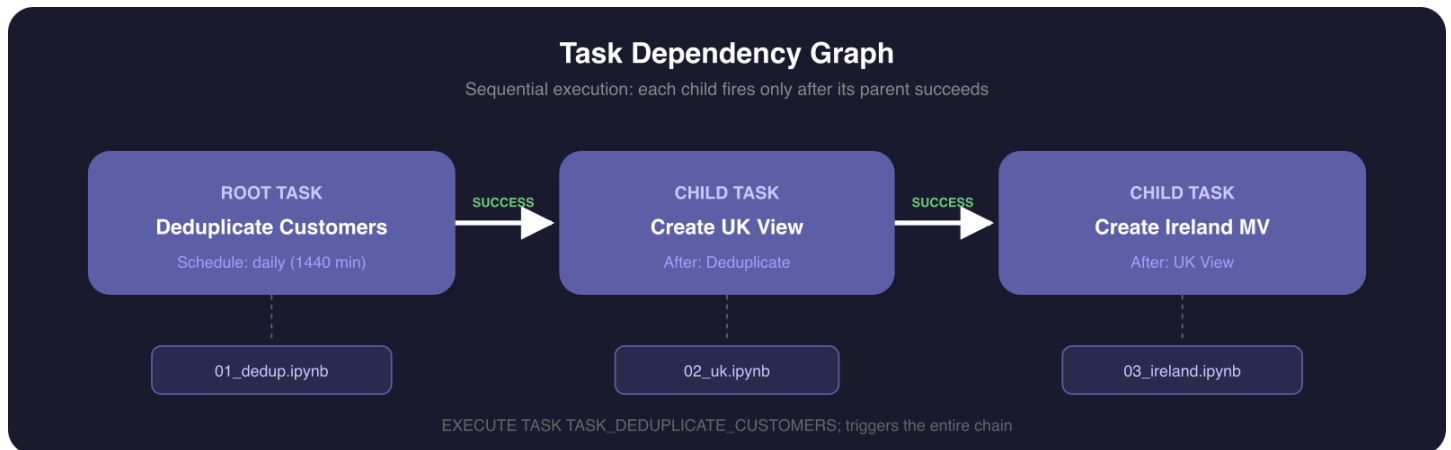


| PIPELINE_LINEAGE |          |                  |                  |             |      |         |
|------------------|----------|------------------|------------------|-------------|------|---------|
| RUN_ID           | NOTEBOOK | SOURCE           | TARGET           | OPERATION   | ROWS | STATUS  |
| demo-001         | 01_dedup | CUSTOMERS        | CUSTOMERS_GOLDEN | CTAS        | 15   | SUCCESS |
| demo-001         | 02_uk    | CUSTOMERS_GOLDEN | CUSTOMERS_UK     | CREATE VIEW | 3    | SUCCESS |

This is custom lineage we control — complementing Snowflake's built-in `ACCESS_HISTORY` and object dependencies. It captures the notebook that created each object and the row counts at execution time.

## 4. Task Orchestration

*"These notebooks don't just run in isolation. They're wired into a dependent task graph."*



The root task runs daily. Each child only fires after its predecessor succeeds. If dedup fails, the downstream views don't get stale data — they simply don't run.

### Manual Trigger

```
EXECUTE TASK LANSDOWNEPARTNERS_DB.CORE.TASK_DEDUPLICATE_CUSTOMERS;
```

### Check Execution

```
SELECT name, state, scheduled_time, completed_time
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  SCHEDULED_TIME_RANGE_START => DATEADD('minute', -5, CURRENT_TIMESTAMP()),
  RESULT_LIMIT => 10
))
WHERE name LIKE 'TASK_CUSTOMERS%' OR name = 'TASK_DEDUPLICATE_CUSTOMERS'
ORDER BY scheduled_time DESC;
```

## 5. Results

```
-- Golden source: all records with master flag
SELECT * FROM LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_GOLDEN LIMIT 10;

-- UK customers only (master records)
SELECT * FROM LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_UK;

-- Ireland customers (materialized for fast access)
SELECT * FROM LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_IRELAND;
```

## 6. Why This Matters

### Key Takeaways



#### Notebooks as Pipeline Steps

Testable  
Readable  
Self-documenting  
Version controlled



#### Dependent Tasks

Guaranteed order  
No race conditions  
Failure isolation  
On-demand trigger



#### Built-in Lineage

Full audit trail  
No external tools  
Row counts + timing  
Source → Target map



#### Golden Source Pattern

Deduplicate once  
Serve many views  
Stable GOLDEN\_ID  
Single source of truth



#### Materialized Views

Pre-computed  
Fast queries  
Auto-refresh  
Dashboard-ready

- Notebooks as pipeline steps — testable, readable, self-documenting
- Dependent tasks — guaranteed execution order, no race conditions
- Built-in lineage — full audit trail without external tools
- Golden source pattern — deduplicate once, serve many views downstream
- Materialized views — pre-computed for fast dashboard queries

*"This pattern scales to any number of downstream consumers. Add a new country? Add a notebook and a task — the lineage tracks it automatically."*

## Quick Reference

| Object                         | Type              | Purpose                     |
|--------------------------------|-------------------|-----------------------------|
| <hr/>                          |                   |                             |
| CUSTOMERS                      | Table             | Raw source data             |
| CUSTOMERS_GOLDEN               | Table             | Deduplicated with GOLDEN_ID |
| CUSTOMERS_UK                   | View              | UK master records           |
| CUSTOMERS_IRELAND              | Materialized View | Ireland master records      |
| PIPELINE_LINEAGE               | Table             | Execution lineage log       |
| TASK_DEDUPLICATE_CUSTOMERS     | Task (root)       | Runs notebook 1 daily       |
| TASK_CUSTOMERS_UK_VIEW         | Task (child)      | Runs notebook 2 after root  |
| TASK_CUSTOMERS_IRELAND_MATVIEW | Task (child)      | Runs notebook 3 after UK    |

## Cleanup

```

ALTER TASK LANSDOWNEPARTNERS_DB.CORE.TASK_DEDUPLICATE_CUSTOMERS SUSPEND;
DROP TASK IF EXISTS LANSDOWNEPARTNERS_DB.CORE.TASK_CUSTOMERS_IRELAND_MATVIEW;
DROP TASK IF EXISTS LANSDOWNEPARTNERS_DB.CORE.TASK_CUSTOMERS_UK_VIEW;
DROP TASK IF EXISTS LANSDOWNEPARTNERS_DB.CORE.TASK_DEDUPLICATE_CUSTOMERS;
DROP MATERIALIZED VIEW IF EXISTS LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_IRELAND;
DROP VIEW IF EXISTS LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_UK;
DROP TABLE IF EXISTS LANSDOWNEPARTNERS_DB.CORE.CUSTOMERS_GOLDEN;
DROP TABLE IF EXISTS LANSDOWNEPARTNERS_DB.CORE.PIPELINE_LINEAGE;

```